

Python for Signal Processing: A Comprehensive Guide

This document serves as a complete reference guide for solving discrete and continuous signal assignments in Python. It heavily focuses on manipulating arrays using NumPy and visualizing signals using Matplotlib, specifically tailored to the operations found in your lab assessments.

1. Core Libraries & Setup

Every assessment starts by importing two main libraries. Python utilizes these highly optimized libraries to handle mathematical computations and graph rendering without needing slow, manual loops.

```
import numpy as np
import matplotlib.pyplot as plt

# A common global constant for discrete bounds
INF = 8
```

2. NumPy Array Manipulation Essentials

Signals are represented as data arrays. Here are the precise NumPy functions you need to generate, index, slice, and filter these arrays.

Creating Arrays

- `np.zeros(size)` : Creates an array of the specified size filled with zeros. Example:
`np.zeros(2 * INF + 1)` .
- `np.zeros_like(x)` : Creates a brand-new array of zeros with the exact same shape and size as array `x` . Crucial for storing shifted or scaled signal results.
- `np.arange(start, stop)` : Generates discrete integer ranges. E.g., `n = np.arange(-INF, INF + 1)` generates the discrete time axis `n` from -8 to 8.
- `np.linspace(start, stop, N)` : Generates continuous ranges. E.g., `t = np.linspace(-4.0, 4.0, 4001)` generates 4001 smooth points for a continuous-time signal.

Array Slicing & Indexing

Slicing is the fastest way to manipulate signals without loops. The syntax is `array[start:stop]`.

- `x[k:]` : Takes elements from index `k` all the way to the end of the array.
- `x[:-k]` : Takes all elements from the start, *up to* the last `k` elements. (Often used when shifting right).
- `x[-k:]` : Takes exactly the *last* `k` elements.
- `x[::-1]` : Reverses the entire array instantly. Used for time reversal `x[-n]`.

Boolean Masking & Conditional Operations

Masking allows you to selectively modify parts of an array based on conditions.

```
# Creating a mask for valid scaling bounds
valid_mask = (k * n >= -INF) & (k * n <= INF)

# Using np.where as an array-wide IF/ELSE statement
val1 = np.where((m1 >= -INF) & (m1 <= INF), x[idx1], 0.0)
```

Math Utilities (For Interpolation & Sub-scaling)

- `np.floor(array)` : Rounds array values down to the nearest integer.
- `np.ceil(array)` : Rounds array values up to the nearest integer.
- `np.clip(array, min, max)` : Restricts values so they do not fall below `min` or exceed `max`. This is vital to prevent "index out of bounds" errors during interpolation.

3. Matplotlib Plotting Syntax

Plotting effectively requires proper axes configuration and choosing the right graph type depending on whether the signal is discrete or continuous.

Discrete Plots (Stem Plot)

For discrete-time signals like `x[n]`, always use `plt.stem()`.

```
def plot_discrete(n, signal, title):
    plt.figure(figsize=(8, 3))

    # Set explicit integer ticks on the X-axis
    plt.xticks(np.arange(-INF, INF + 1, 1))

    # Stem plot for discrete signals
    plt.stem(n, signal)

    plt.title(title)
    plt.xlabel('n (Time Index)')
    plt.ylabel('x[n]')
    plt.grid(True)

    # Save the figure to a file
    plt.savefig('discrete_signal.png')
```

Continuous Plots (Line Plot)

For continuous-time signals like $x(t)$, use `plt.plot()`.

```
def plot_continuous(t, x, y, title):
    plt.figure(figsize=(10, 6))

    # Line plots with colors, labels, and line styles
    plt.plot(t, x, label="Original Signal x(t)", color="blue", linewidth=1.5)
    plt.plot(t, y, label="Scaled Signal y(t)", color="red", linestyle="--")

    plt.title(title)
    plt.xlabel("Time (t)")
    plt.ylabel("Amplitude")
    plt.grid(True)
    plt.legend() # Automatically adds a legend using the 'label' tags
    plt.show()
```

4. Common Signal Operations (Assessment Implementations)

Below are the explicit, vectorized Python implementations for the primary signal operations found in your assignments. These are designed to be efficient and handle boundary conditions properly.

Time Shifting a Discrete Signal

Formula: $y[n] = x[n - k]$. A positive k shifts the signal to the right, and a negative k shifts it to the left. We achieve this purely through array slicing.

```
def time_shift_signal(x: np.ndarray, k: int) -> np.ndarray:
    xr = np.zeros_like(x)
    if k == 0:
        return x.copy()

    if k > 0:
        # Shift Right: Exclude the last k elements of x, put them starting at index k
        xr[k:] = x[:-k]
    else:
        # Shift Left (k is negative): Put the last |k| elements of x at the beginning
        xr[:k] = x[-k:]

    return xr
```

Time Reversal & Even-Odd Decomposition

Time reversal flips the signal: $x[-t]$ or $x[-n]$. Using this, a signal can be decomposed into its Even component $x_e = 0.5(x[n] + x[-n])$ and Odd component $x_o = 0.5(x[n] - x[-n])$.

```
def time_reverse(x: np.ndarray) -> np.ndarray:
    # Instantly reverse the array
    return x[::-1]

def even_odd_decompose(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    xr = time_reverse(x)

    # Vectorized math automatically adds/subtracts element-by-element
    xe = 0.5 * (x + xr)
    xo = 0.5 * (x - xr)

    return xe, xo
```

Time Scaling (Discrete)

When computing $x[kn]$, we drop intermediate values. We iterate or map the new index to ensure it falls within the boundaries of `-INF` and `INF`.

```

def time_scale_signal(x: np.ndarray, k: int) -> np.ndarray:
    res = np.zeros_like(x)
    if k == 0: return res

    for n in range(-INF, INF + 1):
        scaled_n = k * n
        # Check boundaries to prevent out-of-bounds error
        if -INF <= scaled_n <= INF:
            array_index_res = n + INF
            array_index_x = scaled_n + INF
            res[array_index_res] = x[array_index_x]

    return res

```

Interpolation & Sub-scaling (Continuous)

When scaling $y(t) = x(t/k)$ for continuous sampled data, the new required time instances might lie between two original samples. We estimate the missing value by averaging the left and right nearest known points.

```

def interpolate_signal(t_original: np.ndarray, x_original: np.ndarray, t_query:
np.ndarray) -> np.ndarray:
    # Find the time step
    dt = t_original[1] - t_original[0]

    # Find exact fractional indices for the queried time points
    exact_indices = (t_query - t_original[0]) / dt

    # Determine left and right neighbor indices
    left_indices = np.floor(exact_indices).astype(int)
    right_indices = np.ceil(exact_indices).astype(int)

    # Clip the indices to ensure they stay within array bounds
    max_index = len(x_original) - 1
    left_indices = np.clip(left_indices, 0, max_index)
    right_indices = np.clip(right_indices, 0, max_index)

    # Calculate the average of the two neighboring samples
    return 0.5 * (x_original[left_indices] + x_original[right_indices])

def time_scale_continuous(t: np.ndarray, x: np.ndarray, k: int) -> np.ndarray:
    # Evaluate at t/k
    t_query = t / k
    # Request interpolated values for these new time points
    y = interpolate_signal(t, x, t_query)
    return y

```

Pro-Tip for Signal Matching (Phase Change vs. Time Shift):

Sometimes you'll be asked to prove that a time shift n_0 matches a phase shift φ_0 . The formula connecting them in $\cos(\Omega_0 n + \varphi)$ is usually $\varphi_0 = -\Omega_0 \times n_0$. You verify this algorithmically by calculating the Mean Squared Error (MSE) between the two generated arrays: `mse = np.mean((array1 - array2) ** 2)`. If MSE is 0 (or close to 0), the signals perfectly overlap.